

An Approach for Capturing and Documenting Architectural Decisions of Reference Architectures

Milena Guessi
Dept. of Computer Systems
University of São Paulo - USP
São Carlos, Brazil
Email: milena@icmc.usp.br

Flavio Oquendo
IRISA Research Institute
University of South Brittany
Vannes, France
Email: flavio.oquendo@univ-ubs.fr

Elisa Yumi Nakagawa
Dept. of Computer Systems
University of São Paulo - USP
São Carlos, Brazil
Email: elisa@icmc.usp.br

Abstract—During the design of software architectures of software systems, it is widely known the relevance of capturing and documenting architectural decisions, i.e., reasons, implications, justification, and trade-offs related to choices made in these architectures. Thus, a more complete documentation of such architectures can be achieved, contributing therefore to the success of the software systems. In parallel, reference architectures have become an quite relevant element to the successful construction, evolution, and standardization of software systems. Although architectural decisions are also necessary for reference architectures, they have not been sufficiently addressed in the context of these architectures. As a consequence, these architectures become sometimes difficult to be comprehended, harming their dissemination and use. The main contribution of this work is to present a systematic approach for capturing and documenting architectural decisions of reference architectures. In order to observe the viability of our approach, a case study is presented, showing how our approach can enrich the description of these architectures. With this, we intend to bring attention to the importance of such decisions for reference architectures and, as a consequence, contribute to the more complete documentation, dissemination, and effective use of these architectures.

Keywords—*Reference Architecture, Architecture Decision, Architectural Decision, Architecture Description.*

I. INTRODUCTION

Software architectures play an important role in the development life cycle by determining software systems qualities such as maintainability, dependability, and interoperability, as previously mentioned in [1]. But, most of the success of software architectures relies on its architecture description, which encompasses the set of documents expressing the architecture and has several applications, e.g., as basis for system design and development activities, as input to verification and validation automated tools, and as documentation [2].

The documentation of architectural decisions is an important part of architecture descriptions. For example, architecture decisions support the architectural description by recording reasons, implications, justification, and trade-offs for the most relevant architectural choices. As a result, documenting architectural decisions can add a great value to the software architecture in a medium and long term by preventing knowledge vaporization. Different approaches exist for managing architectural knowledge, e.g., pattern-centric (which aims at establishing a shared vocabulary of reusable, abstract solutions); dynamism-centric (which uses ADLs to codify explicit

architectural knowledge into models that can be accessed during runtime); requirements-centric (which intends to bridge the gap between architecture and requirements); and decision-centric (which prevents knowledge vaporization by capturing the reasoning leading to the software architecture) [3].

In the same perspective, we also observe the proposal of reference architectures, which encompass the *knowledge* about how to design software architectures of a given application domain. To do so, reference architectures address business rules, architectural styles, best practices of software development (e.g., architectural decisions, domain constraints, legislation, and standards), and software elements that support development of systems for a given domain [4]. From this, we observe that reference architectures integrate most approaches for architectural knowledge management to provide a single solution and certainly need to document architectural decisions in their architecture descriptions as a prerequisite for their role as effective communication vehicles throughout software development life cycle. Good examples of reference architectures can be found in the literature, such as AUTOSAR (Automotive Open System ARchitecture) [5], for the automotive domain, and UniversAAL [6] and Continua [7], for Ambient Assistent Living (AAL).

Even though AUTOSAR is a very complete architecture, resulting from several years of research and effort, we found out that architectural decisions about the design of the reference architecture are only partially addressed in its documentation, since it lacks rationale and considered trade-offs for example [8]. In fact, to our best knowledge, there is no commonly used approach to address architectural decisions in reference architectures. In this sense, if at least the most important decisions were reported, it would be easier to understand why things are as they are. In addition, since reference architectures differ from concrete software architectures — e.g., they are more generic and often described at a higher abstraction level, have no clear stakeholders, involve more architectural qualities, and have a larger scope — not all approaches used to describe software architectures are also adequate for reference architectures. In this scenario, several approaches for building reference architectures have been proposed [9]–[11]. For example, Muller (2008) provides guidelines for establishing reference architectures. Another study presents a process to establish, represent, and evaluate reference architectures, focusing specifically in improving the separation of concern in such architectures [11]. Angelov et al.

(2012) proposed a framework for classification of reference architectures that aims at promoting analysis and design of reference architectures by their adherence to one of their five types of reference architectures. On the other hand, the first work to introduce the necessity of documenting design decisions in the development of reference architectures is [12]. Thus, it is still necessary to determine how this knowledge should be codified and stored in reference architectures, which should be defined by specific methods or architecture frameworks.

In this scenario, the main contribution of this paper is to present an approach for capturing and documenting architectural decisions during the design of reference architectures. We also present a case study in order to show how our approach can contribute to improve the description of these architectures. As a result, we have observed the importance of such decisions for reference architectures as a means to achieve a more complete documentation, dissemination, and effective use of these architectures. Thus, we also intend this paper motivate investigations for incorporating architectural decisions documentation activity throughout reference architectures' life cycle.

The remainder of this paper is organized as follows. In Section II, we introduce the concept of reference architectures and discuss current approaches for documenting software architectures. In Section III, we present our approach for integrating architectural decisions in the description of reference architectures. In Section IV, we discuss the results from a case study regarding the architectural description of reference architectures. In Section V, we discuss future research perspectives. Finally, we present our final remarks in Section VI.

II. BACKGROUND

Several terms are used to designate software architectures in different abstraction levels. In particular, the term reference model is frequently misused as a synonym for reference architecture. A reference model designates an structure that promotes the understanding on a given domain by sharing a common vocabulary and the parts and its interrelationships without considering implementation details [13]. The OASIS Reference Model for Service Oriented Architectures¹ is an example of reference model. Conversely, reference architectures are less abstract than reference models, as they also provide concrete guidelines for the design of software architectures, such as architectural styles, best practices for software development, and software elements supporting the development of systems. In this context, reference architectures can be derived from the combination of reference models and architecture patterns (see Figure 1). Then, different concrete software architectures can be derived from reference architectures.

The architecture description is the main artifact for accessing software architectures. Given the relevance of architecture descriptions, ISO/IEC/IEEE 42010 standard [2], aims at standardizing and disseminating best practices for the creation of such artifact. As proposed by this standard, the usage of multiple architectural viewpoints is a common

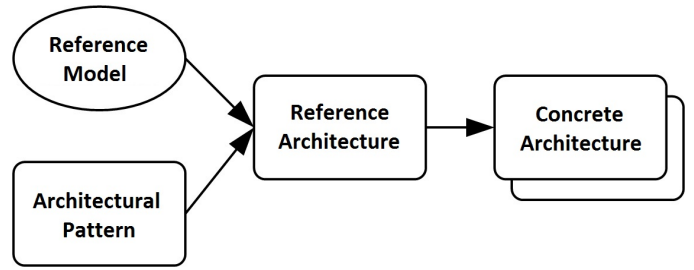


Fig. 1. Relationship among reference model, architectural pattern, reference architecture, and concrete architecture [13]

practice to document architectures of software systems. An architectural viewpoint frames a particular set of concerns from system stakeholders and consists in the conventions for the construction, interpretation, and use of architectural views [2]. Then, an architectural view is a representation of a set of system elements and the relations associated with them which conforms to a specific viewpoint [2]. Several viewpoints, such as logical, runtime, data, and physical viewpoints, have been proposed to represent software architectures as in [14]. In this context, we can also observe the proposal of architecture frameworks, i.e., a set of viewpoints which establish a common practice for describing the software architecture for a particular domain or stakeholder community, such as [15], [16].

One of the main goals of an architectural description is to present sufficient detail about the software architecture in order to enable its analysis against architectural requirements. Based on the success of architectural viewpoints for the representation of software architectures, the use of architectural viewpoints has been also investigated for the representation of reference architectures [?], [17]. Many Architectural Description Languages (ADLs) can be used to describe software architectures, such as Wright [18] and Rapide [19], which are based on formally defined syntax and semantics. But, even though formal ADLs provide automatic mechanisms for validation and verification of architectural models, they can be difficult for non-technical stakeholders to understand. Because of this, semi-formal languages, which combine the rigor of formal languages to the understandability of natural languages, such as UML (Unified Modeling Language) [20] and SysML (System Modeling Language), have also been used for describing software architectures. In particular, SysML introduces a Requirements Diagram which captures the hierarchy among requirements and is useful to guarantee traceability among them, and a Parametric Diagram, which describes constraints to system property values, such as performance, confidence, or weight, and is useful to integrate the specification and models of the system into engineering analysis models.

In another context, architectural knowledge management prevents knowledge vaporization in software architectures. For this, besides having a set of architectural views, which describe the software architecture according to a particular set of concerns, there is also an architecture rationale for significant architectural decisions made. In this sense, significant architectural decisions could be, for example, the selection of a particular concern or viewpoint, definition of a particular abstraction level for viewpoints, or adoption of a particular design pattern. Furthermore, several aspects of an architectural

¹<https://www.oasis-open.org/committees/download.php/19679/soa-rm-cs.pdf>

decision can be relevant, such as their implications to the software architecture design, constraints and rules imposed by them, and also the reasoning that lead to them [21]. In this sense, architectural decisions play an important role for education, reuse, and evolution of software architectures as it is used for sharing expertise and best practices. In this context, reference architectures certainly need to document architectural decisions in their architectural description.

III. INTEGRATING ARCHITECTURAL DECISIONS IN THE DESCRIPTION OF REFERENCE ARCHITECTURES

We include an activity to document architectural decisions into the design process of reference architectures in [11]. The first step for designing the reference architecture is to outline its scope, which consists in selecting the stakeholders of the reference architecture and their concerns. Based on this outline, it is possible to select a set of viewpoints to frame those concerns. After that, the next activity comprises the creation of the selected viewpoints. For this, SysML [22] techniques can be used, since it is an OMG standard and has also been suggested as an evolution of UML. In parallel to all design activities, we advise the documentation of the most relevant architectural decisions. In that sense, they should be kept in a separate repository, which may be directly mapped to other architectural description artifacts (e.g., viewpoints, models). It is relevant to notice that this repository can also be understood as another viewpoint in the reference architecture. We defined these steps inside a loop to indicate that the reference architecture should be built iteratively. At the end of each iteration, there is an evaluation activity that approves the completeness and/or effectiveness of the architectural description to frame the specific set of concerns defined in the beginning of the loop. This evaluation should be developed by relevant stakeholders and developers of the reference architecture. In addition, several evaluation approaches can be applied depending on the selected ADL. If the results of this evaluation are not successful, then all previous activities should be revised to address the identified flaws. In addition, a meeting with the most important stakeholders takes place to decide for more iterations or to accept the architectural description. Subsequently iterations can add more details, concerns, viewpoints, or stakeholders to the architectural description. The steps of our approach are illustrated in Figure 2.

In our approach, we used the architecture framework for architectural decisions proposed by [23]. This framework is composed by four viewpoints, namely: a Decision Detail viewpoint showing information about individual decisions; a Decision Relationship viewpoint showing the relationship between architectural design decisions and their current states in a particular moment in time (e.g., idea, decided, rejected, approved); a Decision Chronology viewpoint showing all versions of an architectural decision, i.e., all states that it assumes over time; and a Decision Stakeholder Involvement viewpoint showing stakeholders' responsibilities in the decision-making process, which is important to personalize architectural knowledge, i.e., documenting who knows what. In their study, Heesch et al. (2012) use UML [20] for most of these viewpoints: the decision detail viewpoint only presents a textual representation (detailed in Table I), the design decision relationship and stakeholders involvement apply the use case diagram,

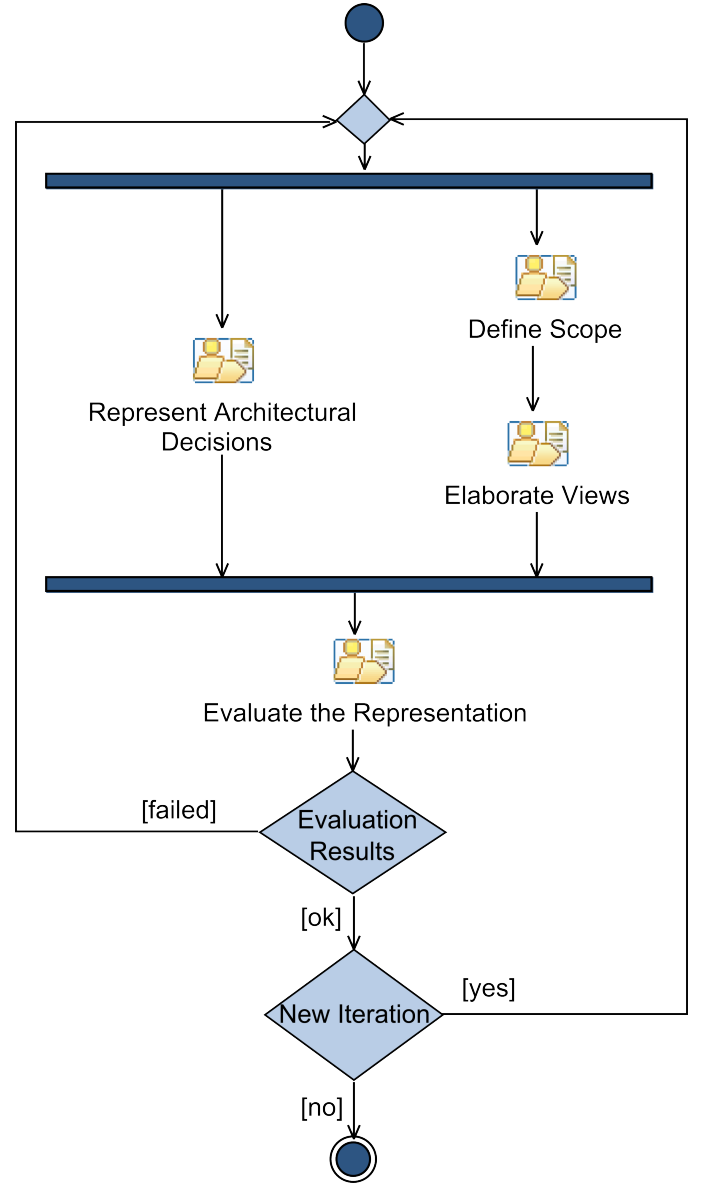


Fig. 2. Approach for documenting reference architectures. In this approach, architectural decisions are documented in parallel to the creation of architecture views

and the chronology viewpoint shows design decisions in a state machine diagram using special stereotypes for states and associations. In our approach, we experimented the use of SysML techniques: the requirements diagram is used in the decision detail viewpoint, the block definition diagram complemented with parametric diagram is used in the design decisions relationship viewpoint, the state machine diagram for the decision chronology viewpoint, and the use case diagram for the decision stakeholders involvement viewpoint.

IV. CASE STUDY

A case study was conducted to evaluate the architectural description of a reference architectures. Our purpose with this study was to access the feasibility and usability of the architectural description to transmit knowledge about the reference architecture design. In order to do so, we created an alternative

TABLE I. TEMPLATE FOR THE DECISION DETAIL VIEWPOINT [23]

Name	A short name of the decision that serves as key in the other viewpoints.
Current state	Indicates the current state of the decision.
Decisions groups	A decision can be associated to one or more groups, which share specific characteristics. Decisions could for instance be grouped by subsystem, architecture team who made the decision, or quality attribute requirements.
Problem/issue	The circumstances under which the architect felt the need to make a decision among one or more alternatives. In other words, the issue addressed by the decision.
Decision	The outcome of the decision. In other templates this element is called solution.
Alternatives	The alternative solutions considered when making the decision.
Related decisions	All decisions that have a relationship to the decision.
Related systems concerns	List of all related concerns tackled in this decision.
History	The history of the described decision. The history contains all state changes, e.g., when the decision was proposed, decided, approved, and so on.

TABLE II. FRAGMENT OF THE DECISION DETAIL VIEW IN OUR CASE STUDY

Name	Abstract from Deployment Context
Problem/issue	Depending on specific application requirements, distribution can take different forms. For a distributed application, the deployment context consists of multiple processors deployed on different nodes that are connected through a network.
Decision	Abstract from Deployment Context
Alternatives	The same instance of the Application Environment subsystem could be deployed on each node, or specific instances are designated to different nodes, e.g., when different types of agents are deployed on different nodes
Related systems concerns	Variability

description for the Situated Multiagent Reference Architecture proposed by Weyns (2006). This reference architecture is composed by: one or more Agents, which are autonomous problem solving entities that can dynamically control their own behavior (i.e., perceive and actuate in the environment); an Application Environment, which is the medium for agents interaction, such as sharing information and coordinating behavior, that has to be designed for each concrete multiagent system; and an Deployment Context, comprised by the hardware, software, and other external resources with which the system interacts (e.g., sensors, actuators, printer, network, database, web service). In his original study, the author included textual explanations about architecture decisions throughout the architecture description. In our architecture description for his reference architecture, we centralized this information in a separate viewpoint, named architecture decisions detail view. We precisely followed the original architectural description in order to guarantee that no errand information was introduced in the new architectural description. As a result, absent information in the original descriptions, such as as history, decisions groups, related decisions, could not be described in the new architecture description. Table II presents a fragment of architecture decisions detail view of the new architectural description.

In total, four subjects participated in our study case. The subjects were separated in two groups and each group analyzed a version of the architecture description. The original version of the architecture description is referred hereinafter to as “original”, while the architecture description including

a specific view for describing architecture decisions is referred to as “new”. All subjects received a link to an on-line questionnaire where they answered a few questions about the reference architecture design including their perception of the architectural description (e.g. facility for accessing a given information, positive and negative aspects of the documentation) and their background on the subjects covered in the reference architecture. In this study, we focus on the subjects’ overall comprehension about the architectural description. Regarding the architecture decision detail view, we specifically inquired our subjects about the reasons for creating a current knowledge repository. We also asked our subjects to grade from 1 (easy) to 4 (difficult) the difficulty level for answering this question. Table III shows the subjects answers and their informed difficulty level. Next, we compare subjects’ performance for accessing this information in both architectural descriptions.

TABLE III. SUBJECTS PERFORMANCE ON THE TEST

Version	New		Original	
Subject	#1	#2	#3	#4
Answer	Correct	Don’t know	Wrong	Wrong
Difficulty	2	4	2	1

From Table III, it is possible to observe that both groups experienced some difficulty to answer this question, although there was a small advantage for the original architecture description. Subjects that analyzed the new architecture description also informed their impressions regarding three aspects of the architecture decision detail view: (i) usefulness, related to the contribution of the view to understand the reference architecture design; (ii) effort, related to the amount of time required to analyze or scan the view for a particular information; and (iii) clarity, related to the amount of time required to understand a particular information in the view. Subjects could grade each aspect from 1 (low) to 4 (high). Table IV presents the subjects answers. We observe that the subjects perception on the usefulness of this view were opposite, as only Subject 1 answered the question correctly.

TABLE IV. SUBJECTS PERCEPTION ON THE ARCHITECTURE DECISION DETAIL VIEW

Version	New	Original
Subject	#1	#2
Usefulness	4	1
Effort	4	3
Clarity	1	2

To better understand the factors that could have played an important role in these results, we asked the subjects to complete a small questionnaire about their personal experience on the topics covered in the architectural description, such as UML, SysML, and Embedded Systems. Table V summarizes subject’s background. We observe that all four subjects present solid knowledge on Software Engineering, also because three of them are doctorate students and one of them is a post-doctorate student in the field. Apart from SysML, we also observe that both groups presented similar knowledge. The lack of a solid SysML did not play a determinant role in the analysis of the architecture decision detail view, because it was textually described in both architecture descriptions. However, this factor should certainly be considered in a future evaluation of this view that uses SysML.

We observed that the participants evaluating the new architectural description presented a better performance than the one

TABLE V. SUBJECTS BACKGROUND ON THE COVERED TOPICS IN THE ARCHITECTURE DESCRIPTION

Version	New		Original	
Subject	#1	#2	#3	#4
UML	4	4	4	3
SysML	2	1	3	2
Embedded System	4	4	4	4
Software Architecture	4	2	3	3
Reference Architecture	4	2	3	3

analyzing the original description. In particular, the subjects evaluating the original description were not able to explain the rationale for an specific architectural decision, although it was explicitly stated in the document. It is relevant to mention that none of the subjects had prior knowledge about this particular reference architecture and they all spent less than a day in their review. It is also worth mentioning that the group evaluating the new architecture description considered the architectural decisions detail view useful but time consuming, as the whole section needed to be linearly scanned. It is important to highlight that this study was supervised by experts to validate the results and conclusions drawn from our results. To avoid biasing the evaluation, subjects were not aware of the existence of a second description.

V. BRIEF DISCUSSION AND PERSPECTIVES OF RESEARCH

Reference architectures have an increasing importance in the development of a set of software systems while architectural decisions provide the necessary information for deriving the reference architecture. Thus, to effectively use the knowledge contained in reference architectures the documentation of architecture decisions should be addressed in parallel to the reference architectures' creation, maintenance, and evolution. In spite of the positive results achieved in this work, it is still necessary to conduct other case studies and/or experiments in order to have more evidences of the relevance of our approach. It is also interesting to use our approach in reference architectures of the industry context.

Based on our findings by conducting this work, as well as our previous experience in the reference architecture research area, we can say that some important research opportunities involving reference architectures and architectural decisions can be drawn, such as:

- Systematizing the documentation of architectural decisions in reference architectures processes by taking design decisions as an intrinsic part of the development of reference architectures. The documentation of architecture decisions since the beginning will prevent knowledge vaporization in reference architectures and will create concrete ways for accessing this knowledge in the future. Based on [25], [26], processes for creating reference architectures, such as [9], [10], [27], should certainly be revised to add specific guidance in this direction;
- Enabling traceability among architecture decisions and other viewpoints in the architecture description. Architecture decisions may crosscut several viewpoints, as they can be related to logical elements (e.g., classes, packages, systems), physical elements (e.g., machines, software installed on these machines, network connections), or functional properties of the reference

architecture. In this sense, enabling traceability from architecture decisions to other artifacts in the architectural description provides interesting insights about the reference architecture internals (e.g., dependencies, considered trade-offs, alternatives). To do so, it is necessary to extend architecture decisions with dependencies, constraints, and relationships;

- Adapting ADLs to document architecture decisions. Currently, architecture decisions mainly informally described, as there is only a textual explanation and their relationship to other artifacts in the architecture description is indirect. In this sense, an ADL can formalize, standardize, and automatize the documentation of architecture decisions. This automatic support ultimately facilitates the recovery of architecture decisions from knowledge repositories, the reuse of architecture decisions in concrete instances of the reference architecture, and the automatic analysis of their dependencies. Nonetheless, UML and SysML still do consider architectural decisions as first class entities. In our case study, we experimented the use of SysML, as it supports the description of requirements and constraints. But, it is still necessary to investigate which are the best techniques for codifying architectural decisions into architecture description elements;
- Introducing architecture decisions variability in reference architectures. The variability in reference architectures concerns the ability of a software artifact built from such architectures to be adapted for a specific context in a preplanned manner [28]. In this sense, adding variability to architecture decisions would help to create an even larger knowledge repository, as all alternatives, dependencies, and options would be registered in the reference architecture. As a result, deriving the reference architecture into concrete software architectures could be resumed to the selection of architecture decisions. For this, it is necessary to investigate in which ways architecture decisions can be tailored from reference architectures to concrete software architectures. In this sense, an approach that adds variability to architectural decisions in software architecture, such as [29], could be investigated for reference architectures.

Therefore, several interesting opportunities regarding architectural decisions in the context of reference architectures exist. However, each aspect of these topics need to be investigated in broader and deeper ways; for instance, through qualitative and quantitative evaluations and experimental studies in industry.

VI. CONCLUSIONS

It is observed that only through a careful documentation of architectural decisions, it will be possible to have a well-documented reference architectures. In this scenario, the main contribution of this work was to propose an approach to deal with architectural decisions of reference architectures. We motivated our proposal with results from a case study. We observed that addressing architectural decisions in the description of reference architectures increases the potential to better communicate and disseminate these architectures. It is

important to say that since reference architectures differ from concrete software architectures, it is necessary to continue the investigation on this topic. In fact, several research lines need to be addressed, yet. As a result, we intend to collaborate to the reference architecture community, as well as software engineering community, by providing means for better capturing, documenting, and disseminating important knowledge contained in reference architectures.

VII. ACKNOWLEDGMENT

This work is supported by São Paulo Research Foundation (FAPESP) (Grant: 2012/24290-5 and 2011/23316-8), CNPq, and Capes.

REFERENCES

- [1] A. I. Wasserman, "Towards a discipline of software engineering," *IEEE Software*, vol. 13, no. 6, pp. 23–31, 1996.
- [2] ISO/IEC/IEEE 42010:2011, *Systems and software Engineering — Architecture Description*, Std., 2011.
- [3] R. Farenhorst and R. C. de Boer, "Knowledge Magament in Software Architecture: State of the Art," in *Software Architecture Knowledge Management Theory and Practice*, M. A. Babar, T. Dingsyr, P. Lago, and H. van Vliet, Eds. Springer, 2009, pp. 21–38.
- [4] E. Y. Nakagawa, P. O. Antonino, and M. Becker, "Reference Architecture and Product Line Architecture: A Subtle But Critical Difference," in *ECSA'2011*, Essen, Germany, 2011, pp. 207–211 (LNCS v. 6903).
- [5] AUTOSAR. (2013) AUTOSAR (AUTomotive Open System ARchitecture). [On-line]. <http://www.autosar.org/> (01/21/2013).
- [6] UniversAAL Project. (2011) The UniversAAL Reference Architecture. [On-line]. <http://www.universaal.org/images/stories/deliverables/D1.3-B.pdf> (01/21/2013).
- [7] Continua Health Alliance, "Continua Health Alliance," [On-line], *World Wide Web*, 2014, in <http://www.continuaalliance.org/> (Access in 02/10/2014).
- [8] E. Y. Nakagawa, M. Becker, and J. C. Maldonado, "A Knowledge-based Framework for Reference Architectures," in *SAC'2012*, Riva del Garda, Italy, 2012, pp. 1197–1202.
- [9] G. Muller. (2008) A Reference Architecture Primer. [On-line]. The Netherlands. <http://www.gaudisite.nl/ReferenceArchitecturePrimerPaper.pdf> (09/30/2013).
- [10] S. Angelov, P. Grefen, and D. Greefhorst, "A framework for analysis and design of software reference architectures," *Information and Software Technology*, vol. 54, pp. 417–431, 2012.
- [11] E. Y. Nakagawa, M. Guessi, J. C. Maldonado, D. Feitosa, and F. Oquendo, "Consolidating a process for the design, representation, and evaluation of reference architectures," in *WICSA'2014*, Sydney, Australia, 2014, pp. 1–10.
- [12] E. Y. Nakagawa, F. Oquendo, and M. Becker, "RAModel: A Reference Model for Reference Architectures," in *WICSA & ECSA'2012*, Helsinki, Finland, 2012, pp. 297–301.
- [13] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley, 2003.
- [14] P. Kruchten, *The Rational Unified Process: An Introduction*, 2nd ed. Addison, 2000.
- [15] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Mereson, R. Nord, and J. Stafford, *Documenting Software Architectures: Views and Beyond*, 2nd ed. Addison-Wesley, 2011.
- [16] P. Kruchten, "Architectural Blueprints - The "4+1" View Model of Software Architecture," *IEEE Software*, vol. 12, no. 6, pp. 42–50, 1995.
- [17] E. Y. Nakagawa and J. C. Maldonado, "Reference architecture knowledge representation: An experience," in *SHARK'2008 at ICSE'2008*, Leipzig, Germany, May 2008, pp. 51–54.
- [18] R. J. Allen, "A Formal Approach to Software Architecture," Ph.D. dissertation, Carnegie Mellon University, 1997.
- [19] D. C. Luckham, "Rapide: A Language and Toolset for Simulation of Distributed Systems by Partial Orderings of Events," in *POMIV'1996*, ser. DIMACS, vol. 29, New Jersey, USA, 1996, pp. 1–25.
- [20] OMG, *Unified Modeling Language v. 2.3*, [On-line], Std., Rev. 2.3, 2010, <http://www.omg.org/spec/UML/2.3/> (01/21/2013).
- [21] J. Bosch, "Software architecture: the next step," in *EWSA'2004*, 2004, pp. 194–199 (LNCS v. 3047).
- [22] OMG, *Systems Modeling Language v 1.2*, [On-line], Std., Rev. 1.2, 2010, <http://www.omg.sysml.org/> (01/21/2013).
- [23] U. van Heesch, P. Avgeriou, and R. Hilliard, "A documentation framework for architecture decisions," *Journal of Systems*, vol. 85, pp. 796–820, 2012.
- [24] D. Weyns, "An Architecture-Centric Approach for Software Engineering with Situated Multiagent Systems," Ph.D. dissertation, Katholieke Universiteit Leuven, Belgium, 2006.
- [25] A. Jansen and J. Bosch, "Software Architecture as a Set of Architectural Design Decisions," in *WICSA'2005*, Pittsburgh, USA, 2005, pp. 1–10.
- [26] P. Kruchten, R. Capilla, and J. C. Dueas, "The Decision View's Role in Software Architecture Process," *IEEE Software*, pp. 36–42, 2009.
- [27] E. Y. Nakagawa, E. F. Barbosa, M. L. Fioravanti, and J. C. Maldonado, "ProSA-RA: A process for the design, representation, and evaluation of aspect-oriented reference architectures," *Journal of Systems and Software*, pp. 1–40, 2011, (in review).
- [28] M. Galster, P. Avgeriou, D. Weyns, and T. Mnnist, "Variability in software architecture: current practice and challenges," *SIGSOFT Software Engineering Notes*, vol. 36, no. 5, pp. 30–32, 2011.
- [29] A. Alebrahim and M. Heisel, "Supporting Quality-driven Design Decision by Modeling Variability," in *QoSA 2012*, Bertinoro, Italy, 2012, pp. 1–6.